

Aplicação de Conversação

FASE 3 – PROJETO DE LABORATÓRIOS DE TELECOMUNICAÇÕES E INFORMÁTICA I

Mestrado Integrado em Engenharia de Telecomunicações e Informática

Grupo 3:

- Leandro Henrique Dantas Alves A82157
- André Martins Almeida A82211
- José Eduardo da Silva Santos A82350



Índice

Índice	2
Índice de figuras	3
1 Introdução	4
2 Tarefa 3.1 – Controlo da ligação lógica.....	5
2.1 Tramas/pacotes	5
2.2 Protocolo de comunicação	5
2.2.1 <i>Stop-and-Wait</i>	6
2.2.2 <i>Sliding Window</i>	7
2.2.3 <i>Checksums</i>	9
2.2.4 CRC.....	9
2.3 Estruturas e técnicas utilizadas.....	10
2.4 Desenvolvimento do código nos módulos Arduino.....	11
2.5 Testes experimentais	12
3 Autoavaliação	13
4 Conclusão	13
5 Referências	14

Índice de figuras

Figura 2.1 - Estrutura de um pacote.....	5
Figura 2.2 - Diagrama temporal da técnica <i>Stop-and-Wait</i>	7
Figura 2.3 - Diagrama temporal da técnica <i>Sliding Window</i> com $W = 3$	8
Figura 2.4 - Cálculo exemplificativo do <i>checksum</i>	9
Figura 2.5 - Cálculo exemplificativo do FCS através de CRC.....	10
Figura 2.6 - Estrutura das tramas para mensagens e confirmações.	10
Figura 2.7 - Fluxograma do código nos módulos Arduino.....	11
Figura 2.8 - <i>Chat</i> entre os utilizadores 0 e 1 visualizado no terminal do utilizador 0.	13

1 Introdução

A tarefa desta terceira fase é de desenvolver o controlo da ligação lógica na comunicação por radiofrequência entre módulos Arduino ligados cada um a um TRF (*transceiver* de radiofrequência) do tipo nRF24L01+.

Os objetivos desta tarefa incluem perceber a noção de trama/pacote, cabeçalho, *payload* e cauda e como representar corretamente a estrutura de um pacote; definir um protocolo de comunicação, especificando e concebendo regras de comunicação, tramas e primitivas de serviço a oferecer à camada superior; definir os tipos de trama e os campos que estas necessitam de conter para proporcionar as funcionalidades desejadas, no âmbito da camada 2 do projeto, para transmissão fiável de dados ponto-a-ponto, com ênfase na deteção e correção de erros; disponibilizar as primitivas às camadas superiores através de APIs (*Application Programming Interfaces*); e efetuar a transferência fiável de um fluxo de dados entre dois módulos Arduino sobre a ligação RF sem fios.

2 Tarefa 3.1 – Controlo da ligação lógica

2.1 Tramas/pacotes

Toda a informação partilhada numa rede está dividida num determinado número de bytes, estes bytes constituem os pacotes/tramas. Cada pacote contém informação que lhe ajudará a chegar ao seu destino, como por exemplo o endereço da origem e o endereço do destinatário, o seu tamanho e identificador. Os dados destes pacotes são transmitidos em protocolos específicos de transmissão ^[1].

Cada pacote é enviado para o seu destino no melhor caminho disponível, um caminho que pode estar a ser usado por todos os outros pacotes numa mensagem ou por nenhum, fazendo com que a rede seja mais eficiente. Esta equilibra o carregamento através de várias peças do equipamento ao milissegundo, depois, se houver um problema com uma peça do equipamento na rede quando estiver a ser transferida uma mensagem, os pacotes podem ser redirecionados à volta do problema, assegurando a entrega da mensagem inteira ^[1].

Cada pacote costuma estar dividido em três partes: cabeçalho, *payload* e cauda, como está representado na Figura 2.1 ^[2].

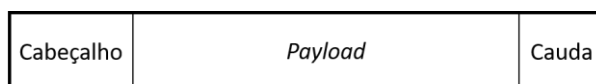


Figura 2.1 - Estrutura de um pacote.

O cabeçalho contém as instruções acerca dos dados do pacote, incluindo o tamanho do mesmo, alguns bits de sincronismo para que o pacote esteja em sintonia com a rede, o identificador do pacote, o protocolo, o endereço do destinatário e o endereço da origem ^[2].

O *payload* (também chamado de corpo ou dados do pacote) contém os dados que o pacote está a enviar ao destinatário. Se o tamanho estiver definido no cabeçalho, o *payload* pode conter informação em branco para estar coerente com o tamanho definido ^[2].

A cauda contém tipicamente alguns bits que informam o recetor que o pacote chegou ao fim. Além disso, pode também incluir um verificador de erros ^[2].

2.2 Protocolo de comunicação

As ligações físicas e a transmissão de sinais, por si só, não garantem uma comunicação de dados fiável entre entidades de diferentes sistemas ^[3].

A troca de dados entre estas entidades deve ser regulada a fim de se criar sincronismo e um contexto comum entre elas, estabelecendo assim um protocolo de comunicação ^[3].

Os protocolos de ligação lógica são responsáveis por ^[3]:

- **Estrutura de tramas** – define o formato da unidade de dados;
- **Configuração e acesso à linha** – considera a topologia e define a disciplina de acesso à linha e a sua duplexidade;
- **Endereçamento** – identifica as interfaces das estações que podem enviar e receber tramas;
- **Controlo de fluxo** – regula a cadência das tramas enviadas, assegurando que a estação que transmite não sobrecarrega a que recebe;
- **Controlo de erros** – deteta erros de transmissão, rejeita os mesmos e executa métodos de recuperação;
- **Controlo da troca de dados** – define como a associação lógica é estabelecida, mantida e terminada.

As técnicas mais comuns de controlo de fluxo são *Stop-and-Wait* (Para-e-Espera) e *Sliding Window* (janela deslizante).

Para o controlo de erros, as técnicas mais comuns são os códigos de blocos (paridade simples ou vertical/longitudinal, usada normalmente pela interface RS-232 (*Recommended Standard*)), somas de controlos (*checksums*) e códigos cíclicos (CRCs – *Cyclic Redundancy Check*). Contudo, nenhuma técnica é 100% fiável.

2.2.1 *Stop-and-Wait*

A técnica *Stop-and-Wait* faz com que após a transmissão de uma trama, o transmissor aguarde a confirmação da sua receção (ACK – *acknowledgement*), enviada pelo recetor, antes de transmitir a trama seguinte ^[3].

Esta técnica tem melhor desempenho quando uma mensagem é fragmentada em poucas tramas de grande dimensão, contudo, grandes dimensões implicam elevadas probabilidades de erros nas tramas, mais ocupação de *buffers* e processadores, e menos desempenho da ligação, principalmente em ligações multiponto ^[3].

O *Stop-and-Wait* não é adequado em fragmentos de pequenas tramas ^[3] pois será perdido muito tempo devido à quantidade excessiva de esperas de ACKs.

Através do diagrama temporal da Figura 2.2, podemos verificar que, após receber uma trama, o recetor envia o ACK imediatamente sempre que estiver disponível para receber mais tramas.

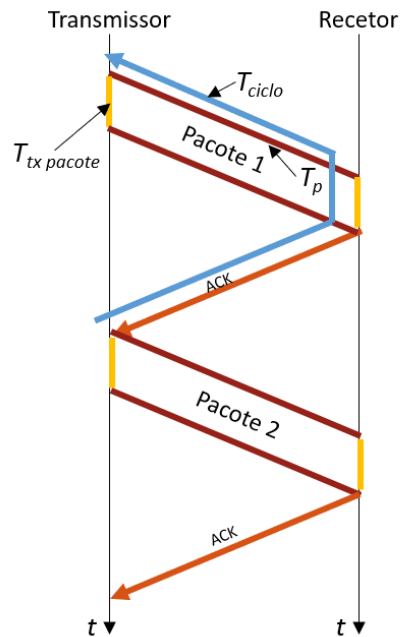


Figura 2.2 - Diagrama temporal da técnica *Stop-and-Wait* [3].

A utilização da ligação (U) pode ser calculada através do tempo útil ($T_{\text{útil}}$) que é igual ao o tempo de transmissão do pacote ($T_{\text{tx pacote}}$) e do tempo de ciclo (T_{ciclo}):

$$U = \frac{T_{\text{útil}}}{T_{\text{ciclo}}} = \frac{T_{\text{tx pacote}}}{T_{\text{tx ack}} + 2 \times T_p + T_{\text{tx pacote}}}.$$

Se forem atribuídos valores nesta equação, por exemplo, $T_{\text{tx pacote}} = 1 \text{ ms}$, $T_p = 10 \text{ ms}$ e $T_{\text{tx ack}} = 0$, pode-se confirmar as desvantagens do *Stop-and-Wait* referidas em cima:

$$U = \frac{T_{\text{útil}}}{T_{\text{ciclo}}} = \frac{1}{2 \times 10 + 1} \approx 4,762\%.$$

Além de controlar o fluxo, esta técnica tem um papel fundamental no controlo de erros, pois não basta ao recetor ter recebido uma mensagem para enviar um ACK, essa mensagem não pode conter erros. Caso contenha, o transmissor terá de reenviar a sua mensagem, este sabe que o tem de o fazer através de duas formas: caso receba um NACK (*negative acknowledgment*) do recetor (pedido explícito para reenviar a mensagem), ou caso não receba um ACK durante o tempo de espera (pedido implícito), o que também é essencial caso o recetor não tenha recebido a mensagem. Esta técnica chama-se retransmissão automática (ARQ – *Automatic Repeat Request*).

2.2.2 Sliding Window

A técnica *Sliding Window* contribui para uma utilização mais eficiente da ligação.

Dada uma abertura da janela W, o emissor pode enviar até W tramas numeradas sem ter que esperar pela confirmação.

Ao contrário da técnica *Stop-and-Wait*, o tempo útil para cada ciclo inclui o tempo de transmissão das W tramas e não de apenas uma, como se pode verificar no diagrama temporal da Figura 2.3.

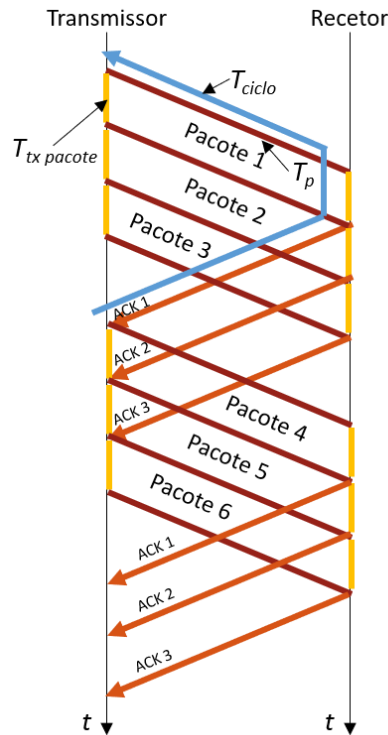


Figura 2.3 - Diagrama temporal da técnica *Sliding Window* com $W = 3$ [3].

A utilização da ligação pode então ser calculada da seguinte maneira:

$$U = \frac{T_{\text{útil}}}{T_{\text{ciclo}}} = \frac{W \times T_{\text{tx pacote}}}{T_{\text{tx ack}} + 2 \times T_p + T_{\text{tx pacote}}}.$$

Se forem atribuídos os mesmos valores do exemplo do *Stop-and-Wait* podemos comparar a utilização de ambas as técnicas:

$$U = \frac{T_{\text{útil}}}{T_{\text{ciclo}}} = \frac{3 \times 1}{2 \times 10 + 1} \approx \mathbf{14,286\%} = 3 \times 4.762\%.$$

Pode-se concluir que a utilização da técnica *Sliding Window* é W vezes maior do que a do *Stop-and-Wait* (até 100%).

O *Sliding Window* também disponibiliza de uma técnica de ARQ chamada *Go-Back-N*. Neste caso, o transmissor não espera pela confirmação da trama anterior para mandar a próxima trama, portanto o recetor ao receber uma trama danificada, envia um NACK com o identificador da trama e vai ignorar as tramas seguintes até que receba a trama corrigida, fazendo com que o transmissor, não só tenha de repetir a transmissão da trama corrigida, mas também de reenviar as tramas ignoradas [4].

2.2.3 Checksums

Na técnica *checksums*, os dados são agrupados em blocos de N bits por linha e por um número variável de linhas e, sendo esses blocos números binários, são somados um a um, adicionando o bit de *overflow* no final de cada soma. Os bits do resultado dessa soma são revertidos, originando o *checksum* que será colocado na cauda da mensagem a enviar ^[3].

Por exemplo, para se calcular o *checksum* da mensagem cujo conjunto de bits é 1110 0110 0110, separada em blocos de 4 bits, segue-se o processo demonstrado na Figura 2.4.

$$\begin{array}{r} 1110 \\ + 0110 \\ \hline 10100 \\ + \quad 1 \\ \hline 0101 \\ + 0110 \\ \hline 1011 \end{array} \quad \begin{array}{l} \text{Checksum} = \\ 0100 \end{array}$$

Figura 2.4 - Cálculo exemplificativo do *checksum* ^[3].

A mensagem a enviar será 1110 0110 0110 **0100**.

O recetor faz o mesmo processo, mas com todos os bits que receber que incluem os dados e o *checksum*. Se o *checksum* calculado tiver o valor zero (1111 = 1011 + 0100), então assume-se que a mensagem foi recebida com sucesso.

2.2.4 CRC

A técnica CRC divide os dados por um número pré-determinado (polinómio gerador) e coloca o resto da divisão num FCS (*Frame Check Sequence*), que será colocado na cauda da mensagem a enviar ^[3].

Por exemplo, para se calcular o FCS da mensagem exemplificada anteriormente através de um polinómio gerador $x^6 + x^5 + x^3 + 1 \rightarrow 1101001$, segue-se o processo demonstrado na Figura 2.5.

$$\begin{array}{r}
111001100110000000 \mid 1101001 \\
\oplus 1101001 \\
\hline
001101000 \\
\oplus 1101001 \\
\hline
0000001110000 \\
\oplus 1101001 \\
\hline
001100100 \\
\oplus 1101001 \\
\hline
00011010 \rightarrow \text{FCS}
\end{array}$$

Figura 2.5 - Cálculo exemplificativo do FCS através de CRC.

O recetor executa a mesma técnica com todos os bits recebidos que incluem os dados e o FCS e se não houverem erros, o FCS calculado terá de ser zero.

2.3 Estruturas e técnicas utilizadas

Nesta fase, as tramas para as mensagens e para as confirmações foram estruturadas nas formas representadas na Figura 2.6.

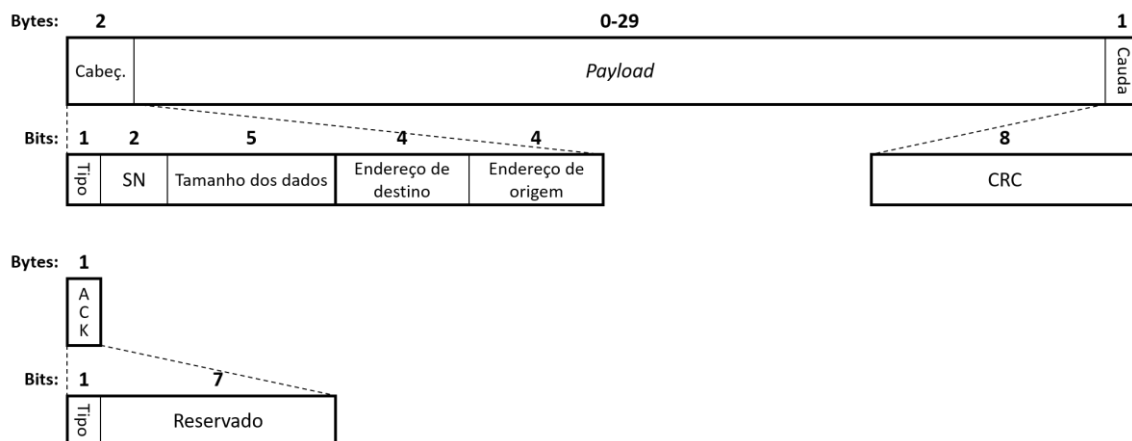


Figura 2.6 - Estrutura das tramas para mensagens e confirmações.

- **Cabeç.** – cabeçalho do pacote;
- **Payload** – dados do pacote;
- **Cauda** – cauda do pacote;
- **Tipo** – tipo do pacote, tem o valor 0 se o pacote for uma mensagem e 1 se for uma confirmação;
- **SN** (*Sequence Number*) – número de sequência do pacote que identifica o mesmo, o seu valor varia entre 0 e 3 sequencialmente;
- **Tamanho dos dados** – tamanho dos dados a enviar que varia entre 0 e 32 bytes;
- **Endereço de destino** – endereço do recetor, podem haver até 16 recetores;
- **Endereço de origem** – endereço do transmissor, podem haver até 16 transmissores;

2.5 Testes experimentais

Os testes executados consistiram na transmissão de pacotes de 32 bytes ($L_{pacote} = 32 \times 8 = 256$ bits), estruturados de acordo com a Figura 2.6, portanto $L_{payload} = 29 \times 8 = 232$ bits, com uma taxa de transmissão de 250 kbps ($R = 250 \times 1024 = 256\,000$ bps), propagando numa distância (d) de 1 m à velocidade da luz ($c = 299\,792\,458$ m/s), além disso, sempre que o recetor recebe uma mensagem corretamente, este responde ao transmissor com um pacote de confirmação de 1 byte ($L_{ack} = 8$ bits) com a mesma taxa de transmissão, distância e velocidade de propagação, usando a técnica *Stop-and-Wait*.

Analisando o esquema da Figura 2.2, podem ser calculados o RTT (*Round-trip Time*), a utilização e os débitos bruto (*throughput*) (S_t) e útil (*goodput*) (S_q):

$$T_{tx\ pacote} = \frac{L_{pacote}}{R} = \frac{256}{256\,000} = 0,001\ s = 1\ ms.$$

$$T_{\acute{u}til} = T_{tx\ pacote} = 1\ ms.$$

$$T_{tx\ ack} = \frac{L_{ack}}{R} = \frac{8}{256\,000} = 0,00003125\ s = 31,25\ \mu s.$$

$$T_p = \frac{d}{c} = \frac{1}{299\,792\,458} \approx 3,3356 \times 10^{-9}\ s \approx 3,336\ ns.$$

$$T_{ciclo} = T_{tx\ ack} + 2 \times T_p + T_{tx\ pacote} = 0,00003125 + 2 \times 3,3356 \times 10^{-9} + 0,001 \approx 0,001031\ s = 1,031\ ms.$$

$$RTT = T_{ciclo} \approx \mathbf{1,031\ ms}.$$

$$U = \frac{T_{\acute{u}til}}{T_{ciclo}} \approx \frac{0,001}{0,001031} \approx \mathbf{96,969\%}.$$

$$S_t = \frac{L_{pacote}}{RTT} \approx \frac{256}{0,001031} \approx 248\,302,619\ bps \approx \mathbf{242,483\ Kbps}.$$

$$S_q = \frac{L_{payload}}{RTT} \approx \frac{232}{0,001031} \approx 225\,024,248\ bps \approx \mathbf{219,750\ Kbps}.$$

Experimentalmente, obteve-se a conversa entre dois utilizadores demonstrada na Figura 2.8.

SN Utilizador

```

0-0: Somebody once told me the wor | Acknowledged (5 ms)
1-0: ld is gonna roll me | Resending | Acknowledged (6 ms).
0-1: I ain't the sharpest tool in
1-1: the shed
2-0: She was looking kind of dumb | Resending | Acknowledged (5 ms).
3-0: with her finger and her thumb | Resending | Acknowledged (6 ms).
0-0: | Acknowledged (8 ms).
2-1: In the shape of an "L" on her
3-1: forehead
1-0: Well the years start coming a | Resending | Acknowledged (5 ms).
2-0: nd they don't stop coming | Resending | Acknowledged (5 ms).
0-1: Fed to the rules and I hit th | Error!
0-1: Fed to the rules and I hit th
1-1: e ground running
3-0: Didn't make sense not to live | Acknowledged (5 ms).
0-0: for fun | Acknowledged (5 ms).

```

Annotations:

- RTT (Redundant Transmission Time) points to the first 'Acknowledged (5 ms)'.
- Confirmação do utilizador 1 points to the first 'Acknowledged (5 ms)'.
- Reenvio da mensagem devido à falta de confirmação do utilizador 1 dentro do *timeout* points to the first 'Resending'.
- Erro recebido points to the 'Error!' message.
- Reenvio da mensagem corrigida pelo utilizador 1 points to the second 'Fed to the rules and I hit th'.

Figura 2.8 - Chat entre os utilizadores 0 e 1 visualizado no terminal do utilizador 0.

Como se pode verificar, obteve-se um RTT de média 5,556 ms, as razões destes valores se afastarem do RTT calculado teoricamente incluem a implementação de um atraso de 2 ms no código para a espera da receção de um pacote, tempos de transmissão da porta série e propagação feita no ar em vez de ser no vácuo. Com este RTT, os débitos podem ser calculados:

$$S_t = \frac{L_{pacote}}{RTT} \approx \frac{256}{0,005556} \approx 46\,080 \text{ bps} = \mathbf{45 \text{ Kbps.}}$$

$$S_q = \frac{L_{payload}}{RTT} \approx \frac{232}{0,005556} \approx 41\,760 \text{ bps} \approx \mathbf{40,781 \text{ Kbps.}}$$

3 Autoavaliação

O Eduardo Santos pesquisou e definiu a estrutura dos pacotes, desenvolveu parte do código para transmissão de pacotes, desenvolveu o fluxograma e fez os cálculos teóricos.

O Leandro Alves implementou no código as técnicas de *Stop-and-Wait* e CRC, parte da transmissão de pacotes, corrigiu erros no código e fez os cálculos experimentais.

O André Almeida desenvolveu o código de receção de pacotes e confirmações da mesma, corrigiu erros no relatório fez os cálculos experimentais.

4 Conclusão

Esta fase ajudou-nos a atingir os objetivos propostos que englobam o controlo da ligação lógica para manter um fluxo correto de dados e evitar erros, detetando-os.

Nesta fase tivemos algumas dificuldades, principalmente em separar bytes do cabeçalho de cada pacote em vários campos com um certo tamanho em bits e no envio do ACK que tinha os bits alterados no lado do recetor.

Além disso, esta fase testou e aprimorou os nossos conhecimentos da matéria da camada de ligação da UC (Unidade Curricular) de Redes de Computadores I.

Concluimos que apesar das dificuldades que surgiram, estas foram ultrapassadas e acreditamos que esta fase foi concluída com sucesso.

5 Referências

- [1] What is a network packet? | HowStuffWorks.
<https://computer.howstuffworks.com/question525.htm>
16/11/2018.
- [2] Network Packet Structure | HowStuffWorks.
<https://computer.howstuffworks.com/question5251.htm>
16/11/2018.
- [3] “Nível de Ligação”, *Slides de Redes de Computadores I* © Universidade do Minho, 2017/2018.
- [4] José Augusto Afonso, “Data Link Layer”.